# V8のArray.prototype.flatを最大約20倍速くするまでと、巨大OSSへの大規模コミットの道のり

西 悠太　/ 株式会社ダイニー

# 自己紹介

**西 悠太** (Nishi Yuta)

**21歳**(重要、important)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです
V8にパッチを送るのも好きです
あと野球も好きです

@riya-amemiya



# そもそも何でこのパッチを出したのか

ある日、Xのタイムラインに「JSCのArray.prototype.flatが速くなる」という投稿が流れてきました

実装を読むと同じ発想がV8にも適用できそうに見えました
約1ヶ月後、私のパッチはV8 14.7にマージされました

```
❯ node -v
v26.11.1

❯ node -p process.versions.v8
14.6.202.34-node.37
```

Chrome、Node.js、Deno、V8を積む
あらゆるランタイムでflatが最大約20倍速くなります

# 最適化メインで話します

- なぜ従来のflatは遅かったのか
- メモリ割り当てを1回に減らす2パス方式
- holeを詰めるflat特有の仕様との格闘
- V8独自言語Torqueでの実装

上記について順を追って解説します

v8-devでの提案からGerritレビュー、マージまでの流れも紹介します

# flatの仕様を理解する

```
> [[1, 2], [3, 4]].flat()
[ 1, 2, 3, 4 ]
> [[[1]], [[2]]].flat(2)
[ 1, 2 ]
```

- `depth` 段までフラットにした新しい配列を返す
- `depth` を省略すると1

# holeの扱い

JavaScriptの配列では、要素が存在しないインデックスのことをholeと呼びます

```
> [1, , 3]
[ 1, <1 empty item>, 3 ]
```

holeは `undefined` と違い、プロパティ自体がない

# ElementsKindとは

V8は配列の中身を
「整数だけ」「浮動小数点を含む」「Holeがある」といった
ElementsKindと呼ばれる内部型で追跡しており、

この型情報を使うと
「数値しか入っていない配列にはサブ配列が存在し得ない」ことが自明になり、
走査そのものを省略できます

# 主要なElementsKind

| | Packed（holeなし） | Holey（holeあり） |
|---|---|---|
| 小さな整数（Smi）だけ | `PACKED_SMI_ELEMENTS` | `HOLEY_SMI_ELEMENTS` |
| 浮動小数点数を含む | `PACKED_DOUBLE_ELEMENTS` | `HOLEY_DOUBLE_ELEMENTS` |
| 文字列やオブジェクトを含む | `PACKED_ELEMENTS` | `HOLEY_ELEMENTS` |

`PACKED_SMI_ELEMENTS` と `PACKED_DOUBLE_ELEMENTS` には、サブ配列もProxyもholeも入らない
この保証を使って最適化する

| | |
|---|---|
| **Torque** | V8の組み込み関数を書く専用言語 |
| **fast path** | 条件がそろったときだけ通る速い経路 |
| **slow path** | どんな入力でも正しく動く従来の経路 |
| **bailout** | 前提が崩れたときにslow pathへ逃げること |

# なぜ従来のflatは
# 遅かったのか

# 従来のflatの実装

1. 長さ0の配列を作る
2. ソース配列を先頭から1個ずつ見る
3. 要素が配列なら、再帰呼び出しでその中を見る
4. 配列でなければ、結果配列の末尾に1個追加する

# 最適化前の全体の流れ

完了

処理　長さ0の配列を作る → 要素を読む → 配列なら再帰呼び出し → **末尾に1個追加** → 足りなければ確保し直す

チェック　構造の変化 24　hole 24　配列か 3　容量 20

入力は [[1, …, 7], [8, , 10, …, 14], [15, …, 21]]




結果配列の容量は 43




確保 2回　コピーし直し 17個　書き込み 20回　再帰呼び出し 3回

# どこが遅かったのか

- 容量が足りなくなるたびに確保し直してコピー
- 汎用のプロパティ書き込みで1個ずつ追加
- サブ配列ごとに再帰呼び出し

## 最適化前

完了

**入力は要素10個のサブ配列6個**



**結果配列の容量は 82**

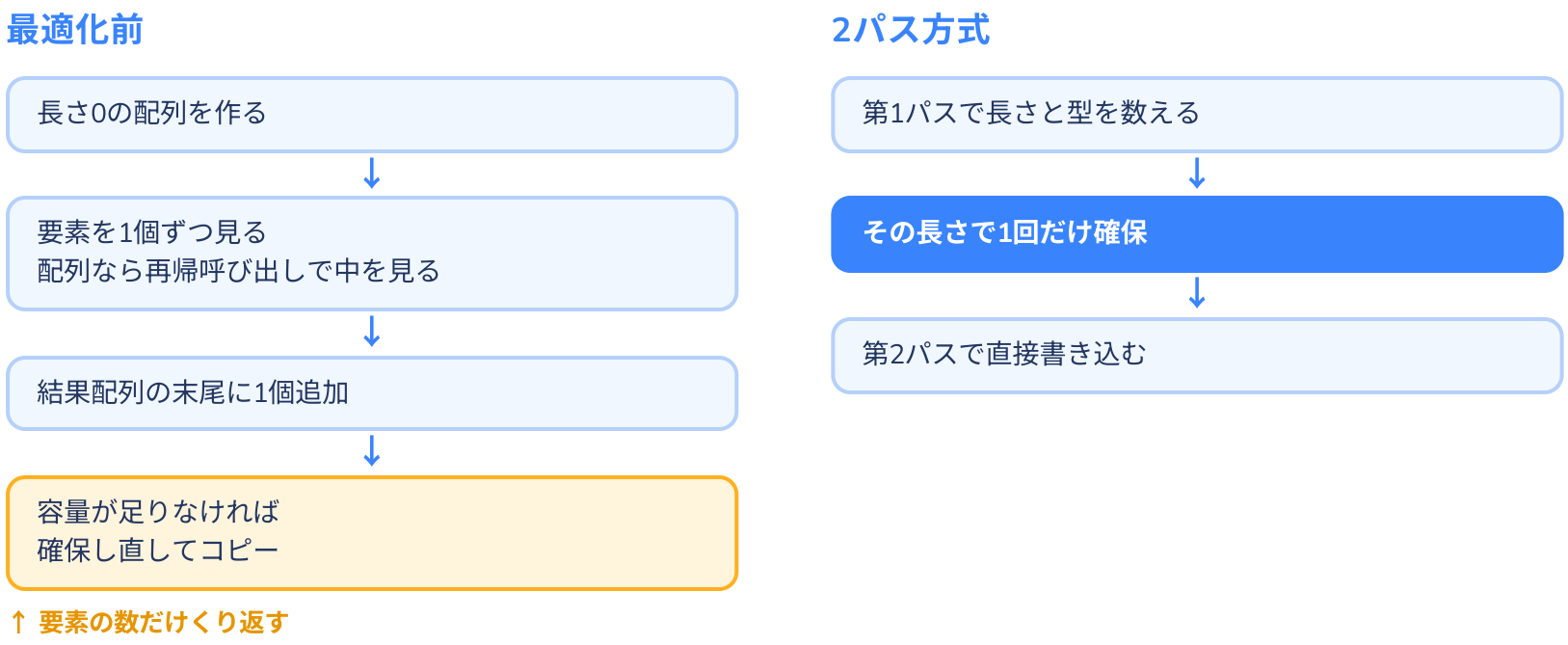
| | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 |
| 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 |
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

確保 3回　書き込み 60回　コピーし直し 60個　再帰呼び出し 6回

新しい容量 = 必要な長さ + 必要な長さの半分 + 16

# メモリ割り当てを
# 1回に減らす2パス方式

# 最適化前と2パス方式




JSCでSosuke Suzukiさんが実装した2パス方式を参考にした

https://github.com/WebKit/WebKit/pull/56035

# 結果配列を何回確保するか

要素10個のサブ配列6個の場合



## 最適化前と2パス方式を同時に動かす

入力は要素10個のサブ配列6個

### 最適化前

完了



確保 3回　書き込み 60回　コピーし直し 60個　再帰呼び出し 6回

### 2パス方式

完了



確保 1回　書き込み 60回　確認 240回

# 数値配列のショートカット

- ソースが `PACKED_SMI_ELEMENTS` か `PACKED_DOUBLE_ELEMENTS` なら、中身は数値だけでholeもない
- だから要素を見ずに `.length` をそのまま結果の長さにする

```
// src/builtins/array-flat.tq
if (sourceKind == ElementsKind::PACKED_SMI_ELEMENTS ||
    sourceKind == ElementsKind::PACKED_DOUBLE_ELEMENTS) {
  return FlattenedLengthResult{length: sourceLength, targetKind: sourceKind};
}
```

`[1, 2, 3, ..., 1024].flat()` の長さ計算はO(1)

```
[[1,2,3], [4,5,6], [7,8,9]].flat()
```

数値だけのサブ配列は、中を読まずに `.length` を足す

**外側の配列は PACKED_ELEMENTS**

| 1 | 2 | 3 | | 4 | 5 | 6 | | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| PACKED_SMI なので +3 | | | | PACKED_SMI なので +3 | | | | PACKED_SMI なので +3 | | |

**結果の長さは 9**

# ElementsKindの追跡

長さを数えながら、結果に入る値の型を3つのフラグで追う

```
// src/builtins/array-flat.tq
if (!IsNumber(element)) {
  seenObject = true;
} else if (!TaggedIsSmi(element)) {
  seenDouble = true;
} else {
  seenSmi = true;
}
```

`[[1], [2], [3]].flat()` の結果は `PACKED_SMI_ELEMENTS`

# 型が混ざった配列

`[[1, 2], [3.5], ["a", "b"]].flat()` の場合

**最適化前**

PACKED_SMI
→ 3.5が来たら確保し直してコピー
PACKED_DOUBLE
→ "a"が来たら確保し直してコピー
PACKED_ELEMENTS

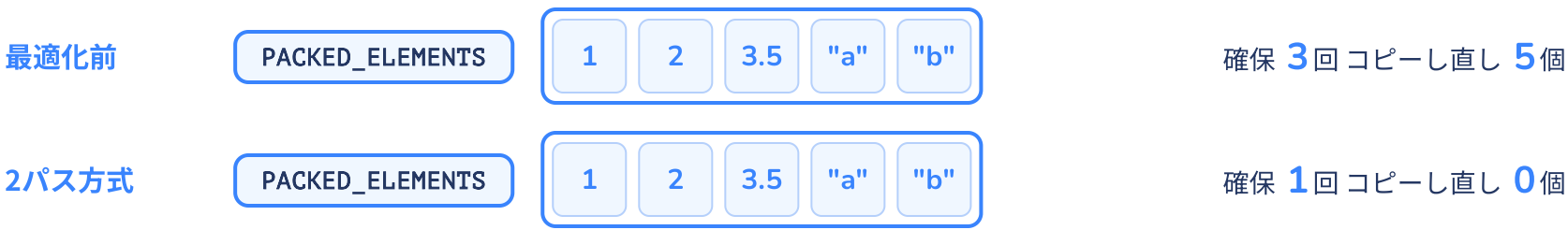
**2パス方式**

第1パスで全部の型を調べる
→ PACKED_ELEMENTS で1回だけ確保

## 型が混ざった配列

完了

入力は `[[1, 2], [3.5], ["a", "b"]]`



# 第2パスはまだ1個ずつコピーしていた

## 2パス方式の第2パス

サブ配列の中へ入る

↓

要素を1個読む

↓

4つの確認
構造の変化、hole、Proxy、書き込み先の範囲

↓

結果配列に1個書く

↑ 要素の数だけくり返す

## バルクコピー

サブ配列の型を1回だけ確認

↓

**memcpyでまとめてコピー**

# 数値のサブ配列はまとめてコピー

第1パスで長さと型が決まっているので、ブロックごと流し込める

**サブ配列 [1, 2, 3] のメモリ**




**↓ memcpy で3個まとめてコピー**

**結果配列のメモリ**



## 2パス方式とバルクコピーを同時に動かす

入力は要素10個のサブ配列6個

### 2パス方式

完了



確保 1回　書き込み 60回　確認 240回

### バルクコピー

完了



確保 1回　memcpy 6回　確認 6回

`TorqueCopyElements` は最終的に `libc` の `memcpy` を呼ぶ

```
// src/builtins/array-flat.tq
if (subArray.map.elements_kind == ElementsKind::PACKED_SMI_ELEMENTS) {
  const srcElements: FixedArray = Cast<FixedArray>(subArray.elements)
      otherwise goto Bailout;
  const srcLen: Smi = Cast<Smi>(subArray.length) otherwise goto Bailout;
  const newIdx: Smi = math::TrySmiAdd(targetIndex, srcLen)
      otherwise goto Bailout;
  if (Convert<intptr>(newIdx) > vector.fixedArray.length_intptr) {
    goto Bailout;
  }
  TorqueCopyElements(
      vector.fixedArray, SmiUntag(targetIndex), srcElements, 0,
      SmiUntag(srcLen));
  targetIndex = newIdx;
  index++;
  continue;
}
```

`PACKED_DOUBLE_ELEMENTS` も同じように `FixedDoubleArray` をコピー

# memcpyできるのは数値配列だけ

| ElementsKind | 中身 | ポインタ | memcpy |
|---|---|---|---|
| `PACKED_SMI_ELEMENTS` | 小さな整数そのもの | なし | できる |
| `PACKED_DOUBLE_ELEMENTS` | 生のdouble | なし | できる |
| `PACKED_ELEMENTS` | 文字列やオブジェクトへの参照 | あり | できない |

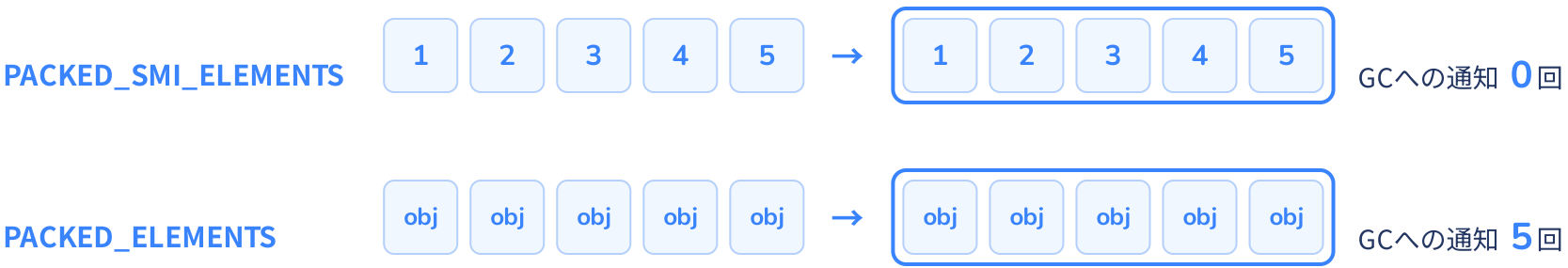
分かれ目はwrite barrier

# write barrier

- V8のGCは世代別なので、どのオブジェクトがどこを指しているかを知っておく必要がある
- ポインタを書くたびにGCへ知らせる仕組みがwrite barrier
- `memcpy` はGCに知らせないので、ポインタを運ぶと使用中のオブジェクトが回収されてuse-after-freeになる

## サブ配列を結果配列へコピー

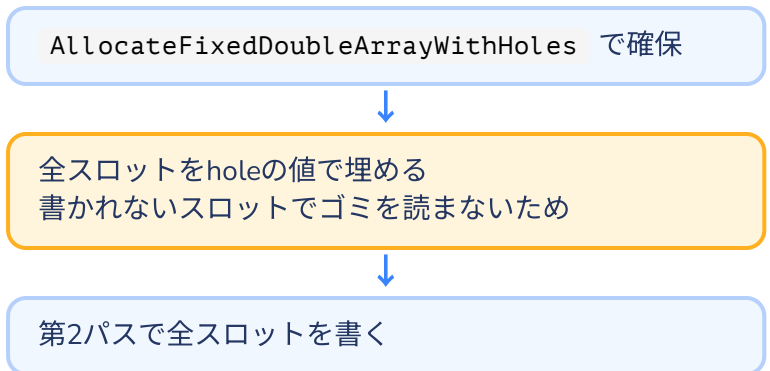
完了



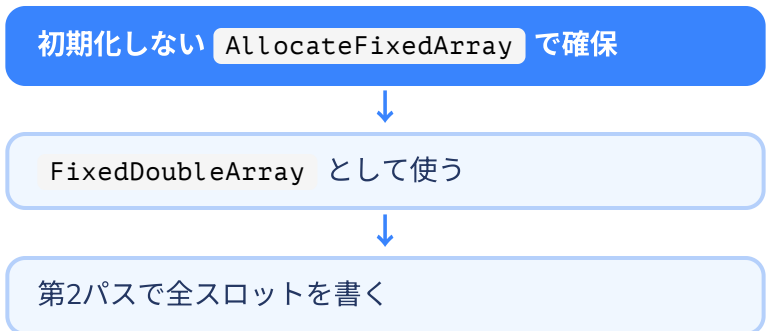
# hole埋めの省略

結果が `FixedDoubleArray` のときは、確保のしかたも変えた

### 2パス方式




### バルクコピー




第1パスで長さを正確に数えているので、第2パスで全スロットが必ず埋まる

# 3つの実装の比較

| | 最適化前 | 2パス方式 | バルクコピー |
|---|---|---|---|
| 結果配列の確保 | 足りなくなるたびに作り直す | **長さを数えて1回だけ** | **1回だけ** |
| サブ配列の処理 | 再帰で中へ入り1個ずつ追加 | 1個ずつ直接書く | **数値配列は `memcpy` でまとめて** |
| 要素ごとの確認 | ある | ある | **数値配列ではない** |
| 速度の目安 | 1x | 約5x | **約20x** |

# holeを詰める
# flat特有の仕様との格闘

```
> [1, , 3].flat()
[ 1, 3 ]
> [1, , 3].map(n => n * 2)
[ 2, <1 empty item>, 6 ]
```

- `map` はholeを残す
- `flat` はholeを飛ばして詰める

# 最適化するうえで厄介なところ

**ソースの .length は 5**




**↓ flat()**

**結果の長さは 3**




- `.length` と結果の要素数が合わない
- 長さを先に出すには、holeを数える必要がある

# HOLEY配列の扱い

HOLEY配列は1個ずつ見て、hole以外を数える

```
// src/builtins/array-flat.tq
let element: JSAny;
try {
  element = fastOW.LoadElementNoHole(index) otherwise FoundHole;
} label FoundHole {
  index++;
  continue;
}
```

- `LoadElementNoHole` はholeを見つけると `FoundHole` へ飛ぶ
- HOLEYなサブ配列は `memcpy` できず、1個ずつコピー
- HOLEY配列でも確保は1回だけ

# holeを詰める

完了

**入力は .length が 10 の配列**




**結果配列の容量は 6**




確保 1回　書き込み 6回　飛ばしたhole 4個

# V8独自言語Torqueでの実装

- 従来の `flat` はTorqueの `FlattenIntoArrayFast` と `FlattenIntoArraySlow`
- その手前に `TryFastFlat` を追加

```
// src/builtins/array-flat.tq
try {
  return TryFastFlat(o, len, depthSmi) otherwise SlowFastPath;
} label SlowFastPath {}

const a: JSReceiver = ArraySpeciesCreate(context, o, 0);
```

条件を満たさなければ従来のslow pathへ

# 明示的スタックによる反復処理

- 再帰で書くと、Torqueコンパイラがスタックオーバーフローで落ちた
- そこでスタックを自前で持つループにした

```
// src/builtins/array-flat.tq
if (stack.length >= kMaxFlatFastStackEntries) goto Bailout;
stack.Push(currentArray);
stack.Push(nextIndex);
stack.Push(currentDepth);
```

- 1段ごとに配列、再開位置、深さの3つを積む
- 積めるのは3072個まで、つまり深さ1024まで

## 明示的スタック

完了

入力は [X, Y].flat(2)　X = ["a", Z]　Z = ["b", "c"]　Y = ["d"]

見ている配列　外側　X　Y　残りの深さ 2

スタック

空

結果　"a"　"b"　"c"　"d"

# どこでbailoutするか

前提がそろっている間だけ速いコードで進み、崩れたらその場でslow pathへ

| 場所 | 確認すること |
| --- | --- |
| fast pathの入口 | 長さがSmiの範囲内か、配列がfast modeか、`Symbol.species` が標準か |
| 走査中 | `Recheck()` で配列の構造が変わっていないか、Proxyがないか |
| 第2パスの最後 | 書き込んだ要素数と、第1パスで数えた長さが一致するか |

# 最適化後の全体の流れ

完了

処理　入口で条件を確認 → 第1パスで長さと型を調べる → 1回だけ確保 → 第2パスで書き込む → 書いた数を確かめる

チェック　入口の条件 1　構造の変化 6　hole 20　配列か 6　書き込み先の範囲 8　長さの一致 1

入力は [[1, …, 7], [8, , 10, …, 14], [15, …, 21]]




結果配列の容量は 20




確保 1回　memcpy 2回　書き込み 6回

# ベンチマーク

- d8（arm64）で、V8 14.6.206とmainを比較
- 要素1,000個のサブ配列1,000個を `flat(2)` して計測
- 8回の最小値

| サブ配列の型 | 最適化前 | 最適化後 | 速度比 |
|---|---|---|---|
| `PACKED_SMI_ELEMENTS` | 6.09 ms | 0.26 ms | 約24x |
| `PACKED_DOUBLE_ELEMENTS` | 7.56 ms | 0.53 ms | 約14x |
| `PACKED_ELEMENTS` | 5.92 ms | 3.87 ms | 約1.5x |
| `HOLEY_SMI_ELEMENTS` | 5.91 ms | 3.45 ms | 約1.7x |
| `HOLEY_DOUBLE_ELEMENTS` | 7.49 ms | 4.66 ms | 約1.6x |
| `HOLEY_ELEMENTS` | 6.12 ms | 3.85 ms | 約1.6x |

# V8に大きな変更を
# 入れるまでの流れ

# まずv8-devで提案する

- 大きな変更は、いきなりパッチを出さない
- 先にレビュアーと方針をすり合わせる
- v8-devに投稿すると、誰かが反応してくれる

Leszek Swirskiさんの返信
「良さそうだね、パッチ出して、議論はそっちでしようか」
https://groups.google.com/g/v8-dev/c/8ROaTLSDXkM

# GerritにCLを出す

- GerritにCLをアップロード
- レビュアーには、変更したファイルのオーナーであるLeszekさんを入れる
- 最初の版はC++のruntime関数で書いていた

# 最初の指摘

- Leszekさん「このfast pathはruntime関数ではなく、Torque/CSAで実装するのがよい」
- runtimeの呼び出しコストを払わずに済む
- V8では普通、fast pathはTorqueかCSAで書き、runtimeを呼ぶのはslow pathだけ
- パッチセット2でTorqueに書き直す

# マージまで

- マージには2人の `Code-Review+1` が要る
- 私にはtry jobもCQも動かす権限がないので、レビュアーに頼む

| ラベル | 動き |
| --- | --- |
| `Commit-Queue+1` | try botでテストだけ回す（dry run） |
| `Commit-Queue+2` | テストが通ったらそのままマージ |

## Chrome 147（V8 14.7）でリリース

[array] Add Torque fast path for Array.prototype.flat

Reviewed-on: https://chromium-review.googlesource.com/c/v8/v8/+/7526287
Reviewed-by: Olivier Flückiger <olivf@chromium.org>
Reviewed-by: Leszek Swirski <leszeks@chromium.org>
Commit-Queue: Olivier Flückiger <olivf@chromium.org>
Commit-Queue: Leszek Swirski <leszeks@chromium.org>
Cr-Commit-Position: refs/heads/main@{#105498}

# ClusterFuzzとの戦い

マージの翌日、GoogleのClusterFuzzがバグを3件発見
buganizer-systemからメールが届く

- crbug 488366773
- crbug 488586038
- crbug 489008235

## 原因は `GetPackedElementsKind` が `HOLEY_DOUBLE_ELEMENTS` を `PACKED_DOUBLE_ELEMENTS` として扱っていたこと

```
// src/builtins/array-flat.tq
macro GetPackedElementsKind(kind: ElementsKind): ElementsKind {
  if (kind == ElementsKind::HOLEY_SMI_ELEMENTS)
    return ElementsKind::PACKED_SMI_ELEMENTS;
  if (kind == ElementsKind::HOLEY_DOUBLE_ELEMENTS)
    return ElementsKind::PACKED_DOUBLE_ELEMENTS;
  if (kind == ElementsKind::HOLEY_ELEMENTS)
    return ElementsKind::PACKED_ELEMENTS;
  return kind;
}

const sourceKind: ElementsKind =
    GetPackedElementsKind(source.map.elements_kind);
```

# なぜクラッシュしたのか

- `HOLEY_DOUBLE_ELEMENTS` はholeを含むので、`.length` と要素数が合わない
- 第1パスで出した長さと実際の要素数がずれる
- `V8_ENABLE_UNDEFINED_DOUBLE` が有効だと、`FixedDoubleArray` にundefinedも入る
- 第2パスの `UnsafeCast<Number>` がundefinedを受け取ってクラッシュ

```
// src/builtins/array-flat.tq
doubleElements.values[targetIndex] =
    Convert<float64_or_undefined_or_hole>(UnsafeCast<Number>(element));
```

# 修正とClose

- `GetPackedElementsKind` を消し、`source.map.elements_kind` を直接見る
- ショートカットは本当にPACKEDなときだけ
- 回帰テストを足し、CLの `Bug:` にissue番号を書く
- マージ後、ClusterFuzzが自動で再評価してClose

# まとめ

- 従来のflatは、結果配列を確保し直しながら1個ずつ追加していた
- 2パス方式で長さと型を先に求め、確保を1回にした
- 数値だけのサブ配列はwrite barrierが要らないので、`memcpy` でまとめてコピー
- holeは第1パスで数えて、第2パスで飛ばす
- Torqueの `TryFastFlat` で実装し、前提が崩れたらslow pathへ
- v8-devで方針を合わせてからGerritに出し、2人の `Code-Review+1` とCQでマージ
- マージ翌日にClusterFuzzが見つけたバグは、回帰テストを足して修正

# 参考資料

- X: x.com/bunjavascript/status/2011898197330075845
- WebKit: WebKit/WebKit#56035
- v8-dev: [Design/Perf] C++ fast path for Array.prototype.flat (2-pass, V8)
- Gerrit: [array] Add Torque fast path for Array.prototype.flat
- Gerrit: [array] Fix flat fast path crash on HOLEY_DOUBLE with undefined
- Gerrit: [array] Add bulk copy shortcuts to Array.flat fast path
- Zenn: Chromium(V8)のArray.prototype.flatを最大約5倍高速化した
- Zenn: Chromium(V8)のArray.prototype.flatをバルクコピーでさらに約5倍高速化した

ご清聴ありがとうございました